

BULK CERTIFICATE GENERATOR

Submission Documentation

API	FastAPI REST API
Database	SQLite + SQLAlchemy
Validation	Pydantic
PDF Generation	ReportLab
Testing	pytest
Server	Uvicorn

A FastAPI-based REST API that accepts a bulk list of recipients and generates individual PDF certificates from a predefined template. The system tracks job progress, stores certificate status in a relational database, and allows generated certificates to be retrieved individually.

Project Highlights

- Bulk certificate generation from a recipient list
- Input validation using Pydantic
- REST API built with FastAPI
- SQLite relational database using SQLAlchemy
- Background processing for certificate generation
- Job status and progress tracking
- Independent certificate failure handling
- Generated PDF certificate retrieval
- Automated tests using pytest
- Interactive API documentation through Swagger UI

1. Technology Stack

Python 3	Core implementation language
FastAPI	REST API framework
SQLAlchemy	ORM / database access
SQLite	Relational database
Pydantic	Request validation
ReportLab	PDF certificate generation
pytest	Automated testing
Uvicorn	Development server

2. Project Structure

```
bulk-certificate-generator/  
├── app/  
│   ├── main.py  
│   ├── database.py  
│   ├── models.py  
│   ├── schemas.py  
│   ├── generator.py  
│   └── worker.py  
├── generated/  
├── test_api.py  
├── README.md  
├── requirements.txt  
├── .gitignore  
└── certificates.db
```

Do not commit venv/, generated/, __pycache__/, .pytest_cache/, or certificates.db to Git.

3. Setup & Running the Application

Create and activate the virtual environment

```
python -m venv venv  
venv\Scripts\activate
```

Install dependencies

```
pip install -r requirements.txt
```

Run the application

```
uvicorn app.main:app --reload
```

API: <http://127.0.0.1:8000>

Swagger UI: <http://127.0.0.1:8000/docs>

4. API Endpoints

Create a Bulk Certificate Job — POST /jobs

The endpoint accepts an event name, certificate date, and recipient list. It returns HTTP 202 Accepted because certificate generation is processed as a background task.

```
{
  "event_name": "Python Workshop 2026",
  "certificate_date": "2026-10-08",
  "recipients": [
    { "name": "Madhuri", "email": "madhuri@example.com" },
    { "name": "Rahul", "email": "rahul@example.com" }
  ]
}
```

```
{
  "job_id": 1,
  "status": "PENDING",
  "total": 2
}
```

Check Job Status — GET /jobs/{job_id}

```
{
  "job_id": 1,
  "status": "COMPLETED",
  "total": 2,
  "successful": 2,
  "failed": 0,
  "progress": "100%"
}
```

Possible statuses: PENDING, PROCESSING, COMPLETED, COMPLETED_WITH_ERRORS.

List Certificates — GET /jobs/{job_id}/certificates

```
[
  {
    "certificate_id": 1,
    "recipient_name": "Madhuri",
    "recipient_email": "madhuri@example.com",
    "status": "SUCCESS",
    "file_path": "generated/certificate_1.pdf",
    "error_message": null
  }
]
```

Retrieve an Individual Certificate — GET /certificates/{certificate_id}

If the certificate was generated successfully, the API returns the generated PDF file.

5. Input Validation

The API validates incoming recipient data before creating a job.

- Invalid email address
- Missing recipient name
- Missing email address
- Empty recipient list

Invalid requests are rejected by FastAPI/Pydantic with HTTP 422 Unprocessable Entity.

6. Certificate Generation

Certificates are generated using ReportLab and a predefined certificate template containing:

- Certificate title
- Recipient name
- Event name
- Certificate date
- Certificate ID

Each generated certificate is saved as:

```
generated/certificate_<certificate_id>.pdf
```

7. Bulk Processing & Failure Handling

Each certificate is processed independently. If generation fails, the certificate is marked as FAILED, the error message is stored, the job's failed counter is increased, and processing continues for the remaining recipients.

Example: if 10 certificates are requested and 1 fails:

```
Total:      10
Successful:  9
Failed:      1
Progress:    100%
Status: COMPLETED_WITH_ERRORS
```

8. Database Design

The application uses two main database tables.

Job

- id
- status
- total
- successful
- failed
- created_at

Certificate

- id

- job_id
- recipient_name
- recipient_email
- status
- file_path
- error_message

Relationship: one Job can contain many Certificate records.

9. Progress Tracking

```
processed = successful + failed
progress = (processed / total) * 100
```

The database is updated after each certificate so that the current state of the job can be queried.

10. Background Processing

FastAPI BackgroundTasks is used to process certificate generation after the API accepts the request.

```
Client
  | POST /jobs
  ▼
FastAPI
  | Create Job + Certificates
  ▼
Database
  | Start background task
  ▼
Worker
  ├── Certificate 1 → SUCCESS
  ├── Certificate 2 → SUCCESS
  └── Certificate 3 → FAILED
    |
    ▼
  Database updated
```

For a production system with a large number of certificates, a durable task queue such as Celery/RQ with Redis would be a better choice.

11. Testing

The project includes automated tests covering the main assignment requirements:

- Job creation
- Input validation
- Certificate generation
- Job progress
- Individual certificate failure

Run tests

```
pytest test_api.py -v
```

Expected result: 5 passed. If a direct certificate download/retrieval test has also been added, the total number of passing tests may be higher.

12. Design Decisions

Why FastAPI?

- Simple REST API development
- Automatic request validation
- Automatic Swagger/OpenAPI documentation
- Easy background task support
- Good developer experience

Why SQLite?

SQLite is simple to set up and does not require a separate database server, making it suitable for this assignment. For production, PostgreSQL would be a stronger choice.

Why SQLAlchemy?

SQLAlchemy provides a clean ORM layer and keeps database operations separate from API logic.

Why BackgroundTasks?

BackgroundTasks keeps the API responsive while certificate files are generated and is sufficient for the scope of the assignment. For high-volume production workloads, a dedicated task queue would be preferred.

13. End-to-End Flow

1. Client sends a bulk recipient list to POST /jobs.
2. FastAPI validates the request.
3. A job is created in the database.
4. Certificate records are created for each recipient.
5. A background task starts processing the job.
6. Each certificate is generated independently.
7. Successful certificates are saved as PDF files.
8. Failed certificates are marked as FAILED with an error message.
9. Job progress is updated after each certificate.

10. Client can check job status using GET /jobs/{job_id}.
11. Client can list generated certificates using GET /jobs/{job_id}/certificates.
12. Client can retrieve an individual PDF using GET /certificates/{certificate_id}.

14. Production Improvements

- PostgreSQL instead of SQLite
- Celery/RQ with Redis for durable background jobs
- Object storage such as S3 for generated certificates
- Authentication and authorization
- Rate limiting
- Structured logging
- Better HTTP error responses
- Job cancellation and retry support
- Database indexes for large datasets
- Docker/container deployment
- Monitoring and health checks
- More comprehensive integration tests

15. Submission Checklist

Before submitting the project, make sure the repository contains:

```
app/  
test_api.py  
README.md  
requirements.txt  
.gitignore
```

Do not submit:

```
venv/  
__pycache__/  
.pytest_cache/  
generated/  
certificates.db
```

Final verification

```
pytest test_api.py -v  
uvicorn app.main:app --reload
```

Then open <http://127.0.0.1:8000/docs> and verify that the API is working.

Conclusion

This project demonstrates a complete bulk certificate generation workflow using FastAPI, SQLAlchemy, SQLite, Pydantic, and ReportLab. It supports validation, background processing, progress tracking, independent failure handling, PDF generation, certificate retrieval, and automated testing.